

بازی زندگی کانوی، مورچه لنگتون و اثبات جامعیت تورینگ

مقدمه

اتوماتای سلولی (Cellular Automata) یکی از مدل‌های محاسباتی جذاب در علوم کامپیوتر است که در آن یک شبکه از سلول‌ها با مجموعه‌ای از قوانین ساده، می‌تواند رفتارهای بسیار پیچیده و غیرقابل پیش‌بینی ایجاد کند. این مدل‌های ریاضی گسسته در نظریه محاسبات و فیزیک نظری کاربردهای فراوانی دارند.

در این پروژه سه مفهوم مهم بررسی و پیاده‌سازی شده است:

- بازی زندگی کانوی (Conway's Game of Life)
- ماشین تورینگ دوبعدی مورچه لنگتون (Langton's Ant)
- اثبات جامعیت تورینگ بازی زندگی با استفاده از گیت‌های منطقی

هدف اصلی پروژه نشان دادن این موضوع است که چگونه قوانین ساده محلی می‌توانند رفتارهای پیچیده و حتی محاسبات عمومی ایجاد کنند. یکی از شگفت‌انگیزترین نتایج اتوماتای سلولی، پدیده «نوظهوری» (Emergence) است؛ جایی که از قوانین محلی بسیار ساده، رفتارهای سراسری فوق‌العاده پیچیده و حتی «جامعیت تورینگ» (Turing Completeness) پدیدار می‌شود.

بخش اول: پیاده‌سازی Conway's Game of Life

2. معرفی Game of Life

بازی زندگی کانوی یک اتوماتای سلولی دوبعدی است که توسط جان کانوی در سال ۱۹۷۰ معرفی شد. در این مدل، صفحه از سلول‌های زنده و مرده تشکیل شده است. هر سلول با توجه به هشت همسایه اطراف خود در نسل بعدی تغییر وضعیت می‌دهد.

3. قوانین Conway

قوانین اصلی به صورت زیر پیاده‌سازی شدند:

سلول زنده:

- اگر تعداد همسایه‌های زنده کمتر از ۲ باشد → مرگ بر اثر تنهایی (Underpopulation)
- اگر تعداد همسایه‌های زنده برابر ۲ یا ۳ باشد → زنده ماندن (Survival)
- اگر تعداد همسایه‌های زنده بیشتر از ۳ باشد → مرگ بر اثر شلوغی (Overpopulation)

سلول مرده:

- اگر دقیقاً ۳ همسایه زنده داشته باشد → ایجاد سلول جدید (Birth)

4. پیاده‌سازی الگوریتم

کلاس اصلی GameOfLife در فایل conway.py پیاده‌سازی شده است.

ویژگی‌های اصلی کلاس:

- نگهداری وضعیت شبکه با استفاده از آرایه‌های NumPy
- اعمال قوانین تکامل به دو روش سلول‌به‌سلول و کانولوشنی
- قرار دادن الگوهای اولیه (Blinker ، Glider ، Glider Gun و...)
- پشتیبانی از دو حالت مرزی (محدود و حلقوی)
- بارگذاری الگو از فایل‌های با فرمت Plaintext و RLE

5. حالت‌های مرزی شبکه

برای بررسی رفتار سیستم، دو مدل مرزی پیاده‌سازی شد:

حالت Finite (محدود):

در این حالت، خارج از محدوده شبکه وجود ندارد و سلول‌های خارج از صفحه همیشه مرده در نظر گرفته می‌شوند. برای مثال، یک سلول در گوشه شبکه تنها ۳ همسایه دارد (به جای ۸ همسایه).

حالت Toroidal / Infinite Wrapping (حلقوی):

در این حالت، لبه‌های شبکه به یکدیگر متصل هستند؛ یعنی خروج از سمت راست معادل ورود از سمت چپ و خروج از بالا معادل ورود از پایین است. این حالت مانند قرار دادن شبکه روی سطح یک کره یا چنبره (Torus) عمل می‌کند.

6. بهینه‌سازی محاسبات

برای افزایش سرعت شبیه‌سازی، دو روش پیاده‌سازی شد:

روش ساده: (Cell-by-cell)

در این روش، هر سلول به صورت جداگانه بررسی می‌شود و تعداد همسایه‌های آن با حلقه‌های تودرتو محاسبه می‌گردد. پیچیدگی زمانی این روش از مرتبه $O(N^2)$ است و برای شبکه‌های بزرگ ($N > 1024$) بسیار کند می‌شود.

روش سریع: (Fast Mode)

در این روش، با استفاده از تابع `scipy.signal.convolve2d`، تعداد همسایه‌های هر سلول به صورت ماتریسی و برداری محاسبه می‌شود. هسته (Kernel) مورد استفاده یک ماتریس 3×3 با مقدار ۱ در تمام خانه‌ها به جز مرکز (که ۰ است) می‌باشد. مزایای این روش عبارتند از:

- سرعت بسیار بیشتر (حدود ۱۰ تا ۱۰۰ برابر سریع‌تر)
- مناسب برای شبکه‌های بزرگ
- کاهش چشمگیر زمان اجرای شبیه‌سازی

7. الگوهای مهم در بازی زندگی

Blinker (نوسانگر):

یک الگوی نوسانی با دوره تناوب ۲ که به صورت افقی و عمودی در نوسان است.

Glider (فضاپیما):

الگویی که به صورت مورب در صفحه حرکت می‌کند. گالیدر با سرعت $c/4$ (یک سلول در هر ۴ نسل) حرکت کرده و شکل خود را در طول حرکت حفظ می‌کند. این ویژگی آن را به یک کاندیدای ایده‌آل برای انتقال سیگنال در محاسبات تبدیل می‌کند. گالیدرها می‌توانند به عنوان حامل‌های اطلاعات در سراسر جهان بازی زندگی حرکت کنند و با برخورد با یکدیگر یا با سایر الگوها، عملیات منطقی را پیاده‌سازی نمایند.

Glider Gun (تفنگ گالیدر):

الگویی که به طور مداوم جریانی از گالیدرها تولید می‌کند. معروف‌ترین نمونه، تفنگ گالیدر گاسپر (Gosper Glider Gun) است که اولین تفنگ گالیدر کشف شده می‌باشد.

8. بارگذاری الگو از فایل

برای بارگذاری الگوهای پیچیده‌تر، تابع `parse_pattern` پیاده‌سازی شده است که از دو فرمت پشتیبانی می‌کند:

فرمت: Plaintext (.cells)

- ساده‌ترین فرمت برای نمایش الگوها
- هر سطر نشان‌دهنده یک ردیف از شبکه است
- کاراکتر `0` نشان‌دهنده سلول زنده و `.` یا `1` نشان‌دهنده سلول مرده است
- خطوط شروع‌شونده با `!` به عنوان کامنت در نظر گرفته می‌شوند

فرمت: RLE (Run-Length Encoded)

- فرمت فشرده‌تری برای الگوهای بزرگ
- از اعداد برای نشان‌دادن تعداد تکرار هر نماد استفاده می‌کند
- `0` نشان‌دهنده سلول زنده، `1` نشان‌دهنده سلول مرده
- `$` نشان‌دهنده پایان ردیف و `!` نشان‌دهنده پایان الگو است
- هدر شامل اطلاعات اندازه الگو می‌باشد `x=width, y=height`

9. تست‌های بخش اول

برای بررسی صحت پیاده‌سازی، چند آزمایش انجام شد:

تست: Blinker

الگوی Blinker یک Oscillator دوره‌ای است که باید با دوره تناوب ۲ نوسان کند.

- نسل ۰: سه سلول زنده به صورت عمودی
- نسل ۱: سه سلول زنده به صورت افقی
- نسل ۲: بازگشت به حالت عمودی

نتیجه 2: PASS: Blinker period = 2

تست قوانین بقا و تولید:

بررسی صحت اعمال قوانین چهارگانه بر روی سلول‌های زنده و مرده.

نتیجه PASS: Survival and Reproduction rules

تست: Boundary

- حالت Finite: PASS

- حالت Toroidal: PASS: Toroidal wrapping works

نتیجه کلی: PART 1a TEST PASSED :

بخش دوم: Langton's Ant :

10. معرفی مورچه لنگتون

مورچه لنگتون یک ماشین تورینگ دوبعدی است که توسط کریس لنگتون در سال ۱۹۸۶ معرفی شد. این سیستم تنها با دو قانون ساده می‌تواند رفتارهای بسیار پیچیده تولید کند. مورچه دارای موقعیت فعلی، جهت حرکت و وضعیت سلول‌های محیط است.

11. قوانین حرکت مورچه

دو رنگ برای سلول‌ها وجود دارد: سفید و سیاه.

اگر روی سلول سفید باشد:

- رنگ سلول تغییر می‌کند به سیاه
- ۹۰ درجه به راست می‌چرخد
- یک خانه جلو حرکت می‌کند

اگر روی سلول سیاه باشد:

- رنگ سلول تغییر می‌کند به سفید
- ۹۰ درجه به چپ می‌چرخد
- یک خانه جلو حرکت می‌کند

12. پیاده‌سازی Langton's Ant

کلاس LangtonsAnt در فایل langton.py پیاده‌سازی شد.

ویژگی‌ها:

- ذخیره موقعیت به صورت (row, column)
- ذخیره جهت حرکت در چهار جهت اصلی (U (Up), R (Right), D (Down), L (Left)
- تغییر رنگ سلول بر اساس قوانین
- چرخش ۹۰ درجه به راست یا چپ
- حرکت یک سلول به جلو با پشتیبانی از مرزهای حلقوی

13. شبیه‌سازی رفتار مورچه

رفتار آشوبناک: (Chaotic Behaviour)

در چند هزار مرحله اول، حرکت‌ها نامنظم هستند و الگوی مشخصی وجود ندارد. مورچه به طور کاملاً بی‌نظم و آشوبناک حرکت کرده و الگوهای متقارن نامنظمی روی صفحه ایجاد می‌کند.

بزرگراه: (The Highway)

بعد از حدود ۱۰,۰۰۰ گام، مورچه ناگهان وارد یک الگوی تکراری و منظم ۱۰۴ گامی می‌شود که در قالب یک بزرگراه مورب، به طور نامتناهی پیشروی می‌کند. این گذار از آشوب به نظم، یکی از جذاب‌ترین جنبه‌های مورچه لنگتون است و مفهوم ظهور (Emergence) را به خوبی نشان می‌دهد.

14. بهبود مورچه با پشتیبانی از چند رنگ

مورچه لنگتون چندرنگی با تعمیم قوانین به بیش از دو رنگ پیاده‌سازی شده است. در این نسخه، هر سلول می‌تواند یکی از چندین رنگ را داشته باشد و قوانین به صورت دیکشنری از رنگ فعلی به (رنگ بعدی، جهت چرخش) تعریف می‌شوند. با تغییر قوانین، الگوهای زیبا و متنوعی ایجاد می‌شوند. برخی از این الگوها شامل تقارن‌های جالب و اشکال هندسی منظم مانند مثلث متقارن هستند.

15. تست Langton's Ant

اجرای نمونه با دستور:

```
python langton_pygame.py --steps 12000 --fps 500
```

نتیجه:

- رفتار اولیه آشوبناک مشاهده شد
- تشکیل Highway پس از حدود ۱۰,۰۰۰ گام تأیید شد

- الگوهای چندرنگی با قوانین مختلف به درستی نمایش داده شدند

بخش سوم: اثبات جامعیت تورینگ با Logic Gates

16. ایده اصلی

یکی از ویژگی‌های مهم Game of Life این است که می‌تواند یک کامپیوتر کامل را شبیه‌سازی کند. برای اثبات این موضوع کافی است نشان دهیم که می‌توان گیت‌های منطقی اصلی را ساخت. در این پروژه، گیت‌های AND و NOT پیاده‌سازی شدند. با ترکیب این گیت‌ها، می‌توان هر مدار دیجیتالی را ساخت که این به معنای جامعیت تورینگ بازی زندگی است.

17. نمایش سیگنال‌ها با Glider

در این بخش، گالیدرها به عنوان بیت‌های منطقی استفاده شدند:

- وجود Glider: نشان‌دهنده بیت ۱
- عدم وجود Glider: نشان‌دهنده بیت ۰

18. پیاده‌سازی AND Gate

توابع `setup_and_gate()` و `run_and_gate()` برای پیاده‌سازی گیت AND طراحی شده‌اند.

ساختار فیزیکی:

- گالیدر A از سمت بالا به پایین حرکت می‌کند
- گالیدر B از سمت چپ به راست حرکت می‌کند
- مسیرها در نقطه‌ای مشخص تقاطع دارند

جدول حقیقت:

A	B	Output
0	0	0
0	1	0
1	0	0
1	1	1

منطق عملکرد:

در پیاده‌سازی، دو گالیدر در مسیر برخورد قرار گرفتند. فقط در حالت حضور هر دو ورودی، برخورد رخ داده و یک بلوک ۲×۲ در منطقه خروجی ایجاد می‌شود که نشان‌دهنده خروجی ۱ است. در سایر حالات، خروجی ۰ می‌باشد.

19. پیاده‌سازی NOT Gate

توابع `setup_not_gate()` و `run_not_gate()` برای پیاده‌سازی گیت NOT طراحی شده‌اند.

ساختار فیزیکی:

- یک گالیدر کنترل ثابت همیشه شلیک می‌شود و به سمت منطقه خروجی حرکت می‌کند
- گالیدر ورودی در صورت فعال بودن، در مسیر گالیدر کنترل قرار می‌گیرد

رفتار:

- **ورودی ۰:** گالیدر کنترل بدون برخورد عبور کرده و به منطقه خروجی می‌رسد ← خروجی ۱
- **ورودی ۱:** گالیدر ورودی با گالیدر کنترل برخورد کرده و هر دو نابود می‌شوند ← خروجی ۰

جدول حقیقت:

Input	Output
0	1
1	0

20. تست Logic Gates

فرمان اجرا:

```
python test_logic_gates.py
```

نتیجه:

AND(0,0) => Output: 0 Expected: 0

AND(0,1) => Output: 0 Expected: 0

AND(1,0) => Output: 0 Expected: 0

AND(1,1) => Output: 1 Expected: 1

PASS: AND Gate

NOT(0) => Output: 1 Expected: 1

NOT(1) => Output: 0 Expected: 0

PASS: NOT Gate

ALL LOGIC GATES TESTS PASSED

21. دلالت‌های جامعیت تورینگ

با داشتن گیت‌های AND و NOT ، می‌توان:

- گیت NAND را ساخت (ترکیب AND و NOT)
 - با استفاده از گیت NAND ، هر تابع بولی دیگر را پیاده‌سازی کرد
 - مدارهای ترکیبی (Combinational Circuits) مانند جمع‌کننده‌ها و ضرب‌کننده‌ها را ساخت
 - مدارهای ترتیبی (Sequential Circuits) مانند فلیپ‌فلاپ‌ها و حافظه‌ها را طراحی کرد
 - در نهایت، یک کامپیوتر کامل را در بازی زندگی پیاده‌سازی کرد
- این اثبات نشان می‌دهد که بازی زندگی کانوی، با وجود قوانین بسیار ساده، به اندازه کامپیوترهای مدرن قدرتمند است و می‌تواند هر الگوریتم قابل‌محاسبه‌ای را اجرا کند.

22. نتیجه‌گیری

در این پروژه نشان داده شد که:

- قوانین ساده Conway می‌توانند رفتارهای پیچیده تولید کنند. با وجود تنها چهار قانون ساده، بازی زندگی قادر به تولید الگوهای متنوعی از جمله نوسانگرها، فضاپیماها و تفنگ‌های گالیدر است.
- مورچه لنگتون نمونه‌ای از یک ماشین تورینگ دوبعدی است. این سیستم با قوانین بسیار ساده، رفتارهای آشوبناک و منظم جالبی از خود نشان می‌دهد و گذار از آشوب به نظم را به خوبی نمایش می‌دهد.
- Game of Life قادر به پیاده‌سازی گیت‌های منطقی است. با استفاده از گالیدرها به عنوان حامل‌های سیگنال، می‌توان گیت‌های AND و NOT را پیاده‌سازی کرد.
- با ساخت گیت‌های AND و NOT می‌توان پایه‌های یک سیستم محاسباتی کامل را ایجاد کرد. این گیت‌ها مجموعه‌ای کامل هستند و با ترکیب آنها می‌توان هر تابع بولی و در نهایت هر محاسبه‌ای را انجام داد.

بنابراین، Conway's Game of Life از نظر محاسباتی Turing Complete است.

این پروژه نشان می‌دهد که چگونه ساختارهای بسیار ساده می‌توانند قابلیت انجام محاسبات عمومی مشابه یک کامپیوتر واقعی را داشته باشند. مفاهیمی مانند نوظهوری، گذار از آشوب به نظم، و جامعیت تورینگ همگی در این پروژه به صورت عملی مشاهده و بررسی شدند.

پیوست: دستورات اجرای پروژه

تست Game of Life

#تست قوانین اصلی با حالت حلقوی

python test_part1a.py --finite false

#تست قوانین اصلی با حالت محدود

python test_part1a.py --finite true

#تست گالیدر با حالت محدود

python test_part1b.py --finite true

#تست گالیدر با حالت حلقوی

python test_part1b.py --finite false

#تست تفنگ گالیدر گاسپر

python test_part1c.py

#تست بارگذاری الگو از فایل

python test_part1d.py

#تست بهینه‌سازی کانولوشن با حالت محدود

python test_part1e.py --finite true

#تست بهینه‌سازی کانولوشن با حالت حلقوی

python test_part1e.py --finite false

اجرای Langton's Ant

اجرای مورچه کلاسیک با تنظیمات پیش فرض

```
python langton_pygame.py
```

اجرای مورچه با ۱۲۰۰۰ گام و سرعت بالا

```
python langton_pygame.py --steps 12000 --fps 500
```

اجرای مورچه چندرنگی

```
python langton_pygame.py --multi-color --steps 2000
```

اجرای مورچه با اندازه شبکه و مقیاس سلول دلخواه

```
python langton_pygame.py --size 300 --cell-scale 4 --steps 10000
```

تست Logic Gates

تست گیت‌های منطقی AND و NOT

```
python test_logic_gates.py
```

نمایشگر بازی زندگی

اجرای نمایشگر بازی زندگی با الگوهای مختلف

```
python pygame_gol.py
```